Предварительный список литературы

Тема: «Разработка и исследование системы наблюдаемости высоконагруженного микросервисного приложения на основе OpenTelemetry».

Автор: Соколов Артем Николаевич.

Группа: 5142704/60801.

Институт: Институт компьютерных наук и кибербезопасности.

Программа: «Киберфизические системы и технологии».

Научный руководитель: Рассказова Альбина Николаевна.

Должность: доцент.

Подразделение: Высшая школа управления кибер-физическими системами.

Учёная степень: кандидат технических наук.

Учёное звание: доцент.

Этап: расширенная предварительная библиография, критические аннотации и сравнение подходов.

Дата подготовки и проверки источников: 29 октября 2026 г.

1. Назначение и состав подборки

Главный вопрос обзора: как выбор способа сбора трассировок влияет на качество диагностики и производительность микросервисного приложения при разной нагрузке?

В список включено 30 источников: 19 рецензируемых научных публикаций [1–19], технический отчёт [20], препринт на рецензировании [21] и девять технических источников [22–30]. Минимум в 12 научных работ выполнен без учёта отчёта, препринта и документации. Версии одной работы не считаются отдельными источниками. Это предварительная тематическая подборка, а не завершённый систематический обзор.

Десять ключевых статей [1–10] разобраны по единой схеме. Остальные источники дополняют сравнение диагностических методов, архитектур и условий измерения. Ранние работы сохранены для самостоятельных задач: анализ логов, динамическое наблюдение, причинные связи и критический путь. Современные измерения расходов представлены публикациями 2025–2026 годов [5–8].

Качество диагностики включает обнаружение нарушения, локализацию причины и обоснование её связи с симптомом. Сохранение аномальной трассы, точность распределения задержек и правильность определения причины являются разными результатами. Их нельзя заменять одним показателем.

2. Методика поиска и чтения

Поиск проводился по сочетаниям distributed tracing sampling overhead diagnosis, OpenTelemetry instrumentation overhead, adaptive sampling, multi-modal root cause analysis, incomplete traces и названиям работ. Проверялись сайты конференций USENIX и ICPE, издателей ACM, IEEE, Springer и Elsevier, университетские и авторские страницы, arXiv и документация OpenTelemetry. Дополнительно проверялись доступные публикации 2026 года; исчерпывающий охват научных баз не заявляется.

Критерии включения: связь с диагностикой или расходами телеметрии, самостоятельная роль, проверяемая библиография и первичное описание метода. Предпочтение отдаётся работам с описанными стендом, нагрузкой, вариантами сравнения и показателями. При ограниченном доступе глубина выводов явно сокращается.

При разборе конкретных статей использованы прочитанные инструкции paper-reading и требования к качеству доказательств из того же репозитория. Навык не установлен локально и не входит в 30 литературных источников. Сначала восстанавливаются вопрос и механизм, затем проверяются эксперимент и границы вывода. Приоритет А означает подробное дальнейшее изучение, Б означает выборочное; это полезность для темы, а не оценка научного уровня.

Численные выводы приводятся с условиями и местом в статье. Результаты авторов отделены от нашей оценки применимости. Сравнение процентов из разных стендов не используется для общего рейтинга. Глубина доступа и нерешённые проверки перечислены в разделе 9.

3. План будущего обзора

3.1. Диагностические задачи и сигналы

Рассмотреть ошибки сервисов, рост задержек, очереди, конкуренцию за ресурсы и распространение отказов. Сопоставить метрики, события в логах и путь запроса; установить, какие наблюдения позволяют проверить предполагаемую причину. Основа: [1, 2, 9, 11–13, 19].

3.2. OpenTelemetry и целостность данных

Описать инструментирование, API, SDK, экспорт и Collector. Разобрать передачу контекста, асинхронные связи и ограничения полноты трассы. Различать стандарт переноса идентификаторов и фактическое покрытие приложения инструментированием. Основа: [10, 16–18, 20, 22, 23, 25–27].

3.3. Способы отбора трассировок

Сравнить ранний вероятностный и поздний отбор, сохранение редких исполнений, адаптивное управление и ретроспективное извлечение. Установить момент решения, доступные сведения и уже понесённые затраты. Основа: [3, 4, 7, 15, 21, 24, 30].

3.4. Расходы и высокая нагрузка

Разделить создание спанов, работу с контекстом, очереди, сериализацию, передачу, обработку и хранение. Рассмотреть насыщение ресурсов и непреднамеренные потери. Сопоставить микротесты с приложением в целом. Основа: [4–8, 14, 28–30].

3.5. Сравнение и постановка исследования

Определить, какие выводы переносятся на OpenTelemetry, а какие зависят от реализации или стенда. Уточнить исследовательские вопросы и требования к эксперименту. Окончательный стек и научная новизна пока не фиксируются.

4. Критические аннотации ключевых статей

4.1. Xu et al.: диагностика по логам [1]

Вопрос и метод. Авторы исследуют обнаружение нарушений по консольным логам. Из шаблонов сообщений, состояний и идентификаторов формируются признаки, затем применяется метод главных компонент.

Эксперимент и показатели. В Darkstar создавались нарушения доступности CPU; отдельно изучались логи HDFS. Проверялись точность разбора, обнаруженные аномалии, ложные срабатывания и масштабируемость обработки (§6.1–6.4). Клиентское время ответа использовалось для проверки нарушения в Darkstar.

Результат и ограничения. Связанные группы сообщений позволяют выявлять отклонения, не выраженные отдельным сообщением об ошибке. Разбор использует исходный код; редкое корректное поведение может выглядеть аномальным. Непрерывный сбор при перегрузке не является предметом эксперимента.

Наша оценка. Приоритет Б. Работа обосновывает ценность логов, но не заменяет современную оценку их сбора через OpenTelemetry. Проверены относящиеся к выводу разделы полного авторского текста.

4.2. Sieve: сокращение метрик [2]

Вопрос и метод. Sieve уменьшает набор метрик фильтрацией и кластеризацией, затем извлекает предсказательные зависимости с помощью проверки причинности по Грейнджеру.

Эксперимент и показатели. Общие свойства проверяются на ShareLatex в кластере из десяти узлов; прикладные сценарии включают автомасштабирование и диагностику OpenStack. Измеряются устойчивость кластеров, число оставшихся метрик, CPU, сеть, хранение и нарушения SLA (§6.1–6.3). Сравниваются сокращённый и исходный сбор.

Результат и ограничения. Авторы демонстрируют уменьшение избыточных измерений и пользу оставшихся зависимостей в выбранных сценариях. Предсказательная связь не доказывает первопричину; полезность сокращения зависит от нагрузки и задачи.

Наша оценка. Приоритет Б. Сокращать можно не только трассы, но нужно проверять сохранение диагностических признаков. Это Sieve для метрик, а не одноимённый отборщик трасс, упоминаемый в TracePicker. Изучены метод и эксперимент авторского текста.

4.3. Sifter: отбор нетипичных трасс [3]

Вопрос и метод. Как сохранять редкие исполнения без ручных признаков неисправности? Sifter строит модель типичных структур по потоку трасс и повышает вероятность отбора плохо описываемых исполнений.

Эксперимент и показатели. Использованы HDFS, DeathStar Social Network и производственный набор; сравнение включает случайный отбор и иерархическую кластеризацию. Проверяются редкие типы, исключения, смена нагрузки и время решения (§7.1–7.6). В §7.6 оно составляет 3–20 мс на трассу в изученных наборах.

Результат и ограничения. Нетипичные исполнения сохраняются с большей вероятностью. Однако редкость не равна неисправности, а локализация причины непосредственно не измеряется. Время работы отборщика нельзя считать добавленной клиентской задержкой.

Наша оценка. Приоритет А. Подходит для сравнения случайной и направленной выборки; расходы обработки входного потока нужно учитывать отдельно. Изучены метод и эксперимент полного авторского текста.

4.4. Hindsight: ретроспективный сбор [4]

Вопрос и метод. Hindsight разделяет локальную запись событий и передачу: данные извлекаются из буферов после программно обнаруживаемого симптома.

Эксперимент и показатели. Использованы DeathStar, HDFS, нагрузка на основе Alibaba и микротесты. Измеряются полнота трасс, время извлечения, задержки и пропускная способность (§6.1–6.4). В MicroBricks с двумя сервисами без дополнительного вычисления максимальная пропускная способность снизилась на 0,9% против 41,7% у сравниваемой конфигурации позднего отбора с перегруженным Collector (§6.4).

Результат и ограничения. Перенос передачи данных уменьшает нагрузку на конвейер. Нужны обнаружимый симптом и извлечение до вытеснения буфера. Различия реализаций и насыщения не позволяют считать проценты чистым эффектом политики отбора.

Наша оценка. Приоритет А. Основной архитектурный контраст для обычного позднего отбора; Hindsight не является штатной конфигурацией Collector. Изучены относящиеся к выводу разделы конференционного полного текста.

4.5. Hammad et al.: инструментирование под нагрузкой [5]

Вопрос и метод. Авторы сравнивают автоматическое инструментирование с приложением без него в облачной среде.

Эксперимент и показатели. Исследованы 70 API двух приложений в AWS и Azure. JMeter создаёт нагрузку с 150, 300, 600 и 1200 одновременными пользователями; предусмотрены прогрев и повторения. Измеряются время ответа, задержка, пропускная способность, ошибки и выбросы; применяются статистические проверки (§3.2 и результаты).

Результат и ограничения. Обнаружены различия показателей, включая ошибки и нестабильность. Описание направления изменения времени ответа неоднозначно; сводные проценты здесь не используются. В §3.2 указано исключение незавершившихся запусков: по нашей оценке, это ограничивает выводы о перегрузке.

Наша оценка. Приоритет Б. Полезна методика нагрузочного сравнения, но качество диагностики отдельно не проверяется. Использованы индексированные разделы издательского текста; сплошное чтение не заявляется.

4.6. Nõu et al.: происхождение расходов [6]

Вопрос и метод. Авторы разделяют расходы настройки, инструментирования и экспорта, сравнивая OpenTelemetry и Elastic APM с вариантом без инструментирования.

Эксперимент и показатели. Использованы отдельные сервисы TechEmpower на нескольких языках и приложения Serverless Benchmark Suite. Измеряются пропускная способность, время выполнения и вклад стадий обработки (§3–6). Для микросервисов оценивается максимальная частота запросов при заданном использовании CPU; для функций рассматриваются задачи разной длительности.

Результат и ограничения. В исследованных микросервисных конфигурациях экспорт составляет существенную часть расходов; для функций результат зависит от задачи и инициализации (§5–6). Отдельный сервис и функция не воспроизводят граф высоконагруженного приложения. Качество диагностики не измеряется.

Наша оценка. Приоритет А. Нужна для раздельного измерения стадий конвейера; величину расходов нельзя переносить на любую конфигурацию. Проверены постановка и результаты полного авторского текста.

4.7. TracePicker: качество сохранённого набора [7]

Вопрос и метод. TracePicker сочетает выделение аномалий, распределение квот между типами трасс и групповой отбор с уменьшением потерь информации.

Эксперимент и показатели. Пять приложений: Train Ticket, Social Network, Online Boutique, Sock Shop и Media; дополнительно используются размеченные наборы Train Ticket. Измеряются сохранение аномальных типов, покрытие типов, баланс, отклонение процентилей задержек, время обработки и отклонение объёма от заданного (§3, §5.1–5.4). Есть сравнение с отборщиками и исключение отдельных модулей.

Результат и ограничения. Текст результатов показывает улучшения характеристик выборки и их конфликт при малых квотах. У базовых методов различается управление долей отбора. Эксперимент не доказывает улучшение локализации причины и не охватывает весь конвейер под нагрузкой.

Наша оценка. Приоритет А. Даёт раздельные критерии качества выборки. Изучены индексированные разделы издательского текста; непосредственно полный текст не прочитан.

4.8. Reichelt et al.: расходы агентов трассировки [8]

Вопрос и метод. Авторы сравнивают агенты Java в расширенном MooBench и исследуют расходы профилированием.

Эксперимент и показатели. Микротест варьирует глубину вызовов и конфигурации инструментирования, сбора и записи; предусмотрены прогрев и повторения. Измеряются добавленное время вызова, зависимость от числа записей и вклад операций; проверяется полнота результата (§2, §4–5).

Результат и ограничения. Различия связаны с метаданными, очередями и деталями реализации. Для некоторых агентов обнаружено неполное сохранение записей, ограничивающее сравнение скорости. Авторы отмечают границы переноса микротеста на длительную конкурентную нагрузку (§6–7).

Наша оценка. Приоритет А. Сравнение расходов следует начинать с проверки одинаковой полезной работы. Рейтинг микротеста не является рейтингом систем наблюдаемости. Изучен текст на сайте ICPE 2026; его пагинация отличается от опубликованной записи.

4.9. Nezha: совместная диагностика [9]

Вопрос и метод. Nezha переводит метрики, логи и трассы в события и сравнивает графовые закономерности нормальной и нарушенной работы для поиска причины внутри сервиса.

Эксперимент и показатели. В OnlineBoutique и TrainTicket вносятся ресурсные нарушения и дефекты кода. OpenTelemetry дополняется идентификаторами трасс и спанов в логах. Оцениваются попадание причины в первые позиции на уровне сервиса и внутри него, сравнение с другими методами и варианты без отдельных сигналов (§5.1–5.3).

Результат и ограничения. Сравнения показывают пользу объединения сигналов в данных условиях. Нужны связанные наблюдения и нормальный период сравнения. Устойчивость диагностики при агрессивном отборе и потерях телеметрии здесь не установлена.

Наша оценка. Приоритет A. Даёт диагностическую задачу для проверки политики сбора. Изучены индексированные разделы первичного PDF; непосредственно файл получить не удалось, численные результаты не цитируются.

4.10. Huye et al.: динамика архитектуры и неполнота [10]

Вопрос и метод. Авторы проверяют, насколько представления о микросервисной топологии и путях запросов соответствуют наблюдениям в Meta.

Данные и показатели. Это наблюдательное исследование, без внесения отказов. Используются история сервисов, сведения о точках входа и трассы Canopy. Измеряются изменение топологии, размеры и структура путей, параллелизм и преждевременное завершение ветвей (§2–4.5).

Результат и ограничения. Топология и исполнения неоднородны, а инструментирование, ограничение скорости и построение трасс искажают наблюдаемую структуру. Данные одной организации и её конвейера ограничивают обобщение; сам способ наблюдения создаёт смещение.

Наша оценка. Приоритет A. Полноту нужно проверять до оценки отбора: отсутствие ветви не доказывает отсутствия вызова. Асинхронные сценарии требуют явных границ запроса. Изучены методика и относящиеся к выводам результаты полного конференционного текста.

5. Сравнение подходов

5.1. Сигналы и результат диагностики

Подход	Что доступно для диагностики	Ограничение
Анализ логов [1]	Связанные события и отклонения поведения	Семантика сообщений и идентификаторы; аномалия не равна причине
Сокращение метрик [2]	Компактное состояние и зависимости	Риск потери признака редкого нарушения
Диагностика по метрикам [11]	Подозрительные сервисы и ресурсы	Модель распространения отклонений
Модели задержек [12, 13, 19]	Зависимости и влияние на общую задержку	Обучение и предположения о взаимодействиях
Совместный анализ [9]	Связь симптома с событием и ресурсом	Согласованность данных и дополнительное инструментирование

Метрики помогают определить момент и масштаб отклонения, трассы связывают его с путём запроса, логи добавляют сведения о событиях. Это наше сопоставление возможностей, а не доказательство обязательной пользы всех сигналов в каждом сценарии. Прямое сравнение точности работ некорректно: различаются неисправности, гранулярность причины и процедуры оценки.

5.2. Момент отбора и стоимость

Способ	Основание решения	Сохраняющиеся расходы и риск
Ранний вероятностный [20, 24, 25]	Контекст начала запроса	Передача контекста; будущий редкий отказ может быть пропущен
Поздний по содержимому [24, 30]	Накопленные спаны	Создание, передача, буферизация; поздние спаны и переполнение
Редкие структуры [3]	Модель типичных исполнений	Обработка потока и модель; редкость не равна неисправности
Квоты и группы [7]	Типы и набор в буфере	Классификация и оптимизация; конфликт покрытия и точности
Адаптивный [15, 21]	Наблюдаемое состояние	Обратная связь; запаздывание реакции и смещение выборки
Ретроспективный [4]	Симптом после локальной записи	Буферы и координация; вытеснение до обнаружения

Поздний отбор уменьшает дальнейшее хранение, но не устраняет уже выполненную работу до отборщика. Ретроспективный подход меняет передачу данных, поэтому его нельзя оценивать только по доле сохранения. Адаптивность означает изменение правила во времени и может сочетаться с разными моментами решения.

5.3. Сопоставимость измерений

Hammad et al. [5] оценивают приложение в облаке, Nõu et al. [6] исследуют отдельные сервисы и функции, Reichelt et al. [8] выделяют расходы Java-агента. Разные единицы анализа не позволяют свести результаты к общему проценту замедления OpenTelemetry.

Для сравнения нужно фиксировать версии, число и размеры спанов, атрибуты, пакетную отправку, размещение Collector, ресурсы и полноту данных. CPU, память и сеть приложения, Collector и хранилища учитываются отдельно. Снижение времени ответа при росте ошибок нельзя автоматически считать улучшением обслуживания.

6. Наши предварительные выводы

Первое: сбор следует оценивать через конкретную диагностическую задачу. Критерии Sifter и TracePicker не заменяют правильность локализации. Нужны известные неисправности и заранее определённая процедура диагностики [3, 7, 9].

Второе: одинаковая доля сохранения не означает одинаковые расходы. Решение принимается на разных стадиях, трассы имеют разный размер. Нужны сравнения при одинаковом объёме сохранённых данных и отдельно при одинаковом бюджете ресурсов [4, 6, 8, 30].

Третье: потери при перегрузке нужно отделять от намеренного отбора. Они могут возникать до отборщика и зависеть от глубины вызовов, маршрутизации и очередей. Вариант «сохранять всё» становится эталоном только после проверки полноты [8, 10, 28–30].

Четвёртое: совместная диагностика требует согласованности сигналов. Сохранённой трассы с ошибкой может быть недостаточно без связанных логов и метрик в нужном временном окне. Nezha задаёт направление проверки, но не доказывает устойчивость к любому сокращению данных [9].

Пятое: в этой подборке недостаточно прямых сопоставлений, одновременно измеряющих локализацию причины и расходы всего конвейера при изменении нагрузки. Это пробел текущей доказательной базы, а не утверждение об отсутствии таких работ вообще. RADAR [21] расширяет адаптивное направление, но его статус пока не позволяет считать результат рецензируемым подтверждением.

7. Исследовательские вопросы

- ИВ1. Как ранний и поздний отбор влияют на сохранность свидетельств и правильность локализации при одинаковом объёме сохранения?
- ИВ2. Как рост нагрузки меняет расходы приложения, очереди Collector и непреднамеренные потери спанов?
- ИВ3. Когда сохранение редких и ошибочных трасс улучшает диагностику, а когда только изменяет статистику выборки?
- ИВ4. Как асинхронные вызовы и неполные трассы влияют на выбранную процедуру диагностики?
- ИВ5. Даёт ли адаптивное управление преимущество после учёта расходов и времени реакции контроллера?

Рабочая гипотеза: при ограниченном бюджете поздний отбор может лучше сохранять известные признаки отказа, но его полезность может снижаться при насыщении конвейера. Это авторская гипотеза, а не результат перечисленных работ.

Минимальные требования к будущему эксперименту:

- Приложение без инструментирования; инструментирование без экспорта для разделения затрат; полный сбор после проверки потерь; сравниваемые политики отбора.
- Одинаковые приложение, версии и ресурсы; прогрев, повторения и чередование конфигураций. Нагрузку от устойчивой работы до насыщения определить предварительным измерением.
- Неисправности с известной причиной: ошибка сервиса, задержка зависимого вызова, конкуренция за CPU, накопление очереди. Асинхронный сценарий включить при его наличии в приложении.
- Независимый клиентский учёт успешной пропускной способности, ошибок и p95/p99. Выборка трасс не должна определять одновременно метод и критерий оценки.
- Раздельно измерять сохранность свидетельств, ранжирование причины, время диагностики, CPU, память, сеть и потери по стадиям.
- Сопоставить одинаковый объём сохранения и отдельно одинаковый бюджет инфраструктуры; проверить поздние спаны, переполнение и согласованность сигналов.

Выбор приложения, языков, SDK, хранилища и диагностического алгоритма остаётся отдельным этапом. DeathStarBench [14] является кандидатом для стенда; работа о Meta [10] ограничивает перенос его результатов на промышленную архитектуру.

8. Предварительный список литературы

8.1. Рецензируемые научные публикации

1. Xu W., Huang L., Fox A., Patterson D., Jordan M. I. Detecting Large-Scale System Problems by Mining Console Logs // Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles (SOSP). 2009. P. 117–132. DOI: 10.1145/1629575.1629587. Авторский текст.

Роль: анализ логов; ключевая статья, §4.1.

2. Thalheim J., Rodrigues A., Akkus I. E., Bhatotia P., Chen R., Viswanath B., Jiao L., Fetzer C. Sieve: Actionable Insights from Monitored Metrics in Distributed Systems // Proceedings of the 18th ACM/IFIP/USENIX Middleware Conference. 2017. P. 14–27. DOI: 10.1145/3135974.3135977. Авторский текст.

Роль: сокращение метрик; ключевая статья, §4.2.

3. Las-Casas P., Papakerashvili G., Anand V., Mace J. Sifter: Scalable Sampling for Distributed Traces, without Feature Engineering // Proceedings of the ACM Symposium on Cloud Computing (SoCC). 2019. P. 312–324. DOI: 10.1145/3357223.3362736. Авторский текст.

Роль: отбор нетипичных исполнений; ключевая статья, §4.3.

4. Zhang L., Xie Z., Anand V., Vigfusson Y., Mace J. The Benefit of Hindsight: Tracing Edge-Cases in Distributed Systems // Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI). 2023. P. 321–339. Страница конференции. Полный текст.

Роль: ретроспективный сбор; ключевая статья, §4.4.

5. Hammad Y., Al-Said Ahmad A., Andras P. An Empirical Study on the Performance Overhead of Code Instrumentation in Containerised Microservices // Journal of Systems and Software. 2025. Vol. 230. Article 112573. DOI: 10.1016/j.jss.2025.112573. Университетская запись.

Роль: инструментирование под нагрузкой; ключевая статья, §4.5.

6. Nõu A., Talluri S., Iosup A., Bonetta D. Investigating Performance Overhead of Distributed Tracing in Microservices and Serverless Systems // Companion of the 16th ACM/SPEC International Conference on Performance Engineering. 2025. P. 162–166. DOI: 10.1145/3680256.3721316. Авторский текст.

Роль: разложение расходов; ключевая статья, §4.6.

7. Xie S., Wang J., Li M., Chen P., Xuan J., Li B. TracePicker: Optimization-Based Trace Sampling for Microservice-Based Systems // Proceedings of the ACM on Software Engineering. 2025. Vol. 2, FSE. Article FSE081. P. 1802–1823. DOI: 10.1145/3729351.

Роль: качество выборки; ключевая статья, §4.7.

8. Reichelt D. G., Yang S., Hanson M., Hasselbring W. Benchmarking the Overhead of Distributed Tracing Agents // Proceedings of the 17th ACM/SPEC International Conference on Performance Engineering (ICPE). 2026. P. 147–161. DOI: 10.1145/3777884.3797004. Конференционный текст. Университетская запись.

Роль: расходы агентов и полнота записей; ключевая статья, §4.8.

9. Yu G., Chen P., Li Y., Chen H., Li X., Zheng Z. Nezha: Interpretable Fine-Grained Root Causes Analysis for Microservices on Multi-modal Observability Data // Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE). 2023. P. 553–565. DOI: 10.1145/3611643.3616249. Первичный PDF проекта.

Роль: совместная диагностика по трём сигналам; ключевая статья, §4.9.

10. Huye D., Shkuro Y., Sambasivan R. R. Lifting the veil on Meta’s microservice architecture: Analyses of topology and request workflows // Proceedings of the 2023 USENIX Annual Technical Conference. 2023. P. 419–432. Страница конференции. Полный текст.

Роль: динамическая топология и неполнота; ключевая статья, §4.10.

11. Wu L., Tordsson J., Elmroth E., Kao O. MicroRCA: Root Cause Localization of Performance Issues in Microservices // NOMS 2020: IEEE/IFIP Network Operations and Management Symposium. 2020. P. 1–9. DOI: 10.1109/NOMS47738.2020.9110353. Авторский проект.

Роль: локализация по связям прикладных симптомов и ресурсных метрик. Проверены первичная аннотация и описание проекта; подробный эксперимент требует чтения полного текста.

12. Gan Y., Liang M., Dev S., Lo D., Delimitrou C. Sage: Practical and Scalable ML-Driven Performance Debugging in Microservices // Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS). 2021. P. 135–151. DOI: 10.1145/3445814.3446700. Авторский текст.

Роль: модель зависимостей и контрфактическая оценка задержек для диагностики. Выполнено первичное ознакомление; применимость предположений о вызовах проверять отдельно.

13. Gan Y., Zhang Y., Hu K., Cheng D., He Y., Pancholi M., Delimitrou C. Seer: Leveraging Big Data to Navigate the Complexity of Performance Debugging in Cloud Microservices // Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS). 2019. P. 19–33. DOI: 10.1145/3297858.3304004. Авторский текст.

Роль: сочетание трасс RPC и аппаратного мониторинга для предсказания нарушений; позволяет отделить предупреждение от диагностики после события. Выполнено первичное ознакомление.

14. Gan Y., Zhang Y., Cheng D. et al. An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems // Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS). 2019. P. 3–18. DOI: 10.1145/3297858.3304013. Авторский текст.

Роль: DeathStarBench как кандидат для воспроизводимого стенда; источник не доказывает эффективность отбора. Выполнено первичное ознакомление.

15. Song Y., Li Y., Wu S., Yang H., Li W. ASTracer: An Efficient Tracing Tool for HDFS with Adaptive Sampling // Network and Parallel Computing (NPC 2019). Lecture Notes in Computer Science. 2019. Vol. 11783. P. 107–119. DOI: 10.1007/978-3-030-30709-7_9.

Роль: адаптивные отборщики и автоматическая настройка в HDFS. Проверена издательская аннотация; количественное сравнение с OpenTelemetry доступным материалом не обосновано.

16. Mace J., Roelke R., Fonseca R. Pivot Tracing: Dynamic Causal Monitoring for Distributed Systems // Proceedings of the 25th Symposium on Operating Systems Principles (SOSP). 2015. P. 378–393. DOI: 10.1145/2815400.2815415. Авторская страница.

Роль: изменение наблюдений во время работы и связывание событий через причинный порядок. Методический фон, не замена современным нагрузочным измерениям; выполнено первичное ознакомление.

17. Kaldor J., Mace J., Bejda M. et al. Canopy: An End-to-End Performance Tracing And Analysis System // Proceedings of the 26th Symposium on Operating Systems Principles (SOSP). 2017. P. 34–50. DOI: 10.1145/3132747.3132749. Авторский текст.

Роль: промышленный конвейер от трасс к агрегированным признакам, разделение сбора и анализа. Выполнено первичное ознакомление с архитектурой.

18. Fonseca R., Porter G., Katz R. H., Shenker S., Stoica I. X-Trace: A Pervasive Network Tracing Framework // Proceedings of the 4th USENIX Symposium on Networked Systems Design & Implementation (NSDI). 2007. P. 271–284. Страница и полный текст.

Роль: связывание событий через уровни системы, историческая основа передачи контекста. Проверена конференционная запись; для оценки текущих расходов не используется.

19. Chow M., Meisner D., Flinn J., Peek D., Wenisch T. F. The Mystery Machine: End-to-end Performance Analysis of Large-scale Internet Services // Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2014. P. 217–231. Страница и полный текст.

Роль: восстановление модели исполнения из событий и анализ влияния компонентов на задержку. Проверено первичное описание метода; причинные выводы требуют отдельной оценки.

8.2. Технический отчёт и препринт

20. Sigelman B. H., Barroso L. A., Burrows M., Stephenson P., Plakal M., Beaver D., Jaspan S., Shanbhag C. Dapper, a Large-Scale Distributed Systems Tracing Infrastructure. Google, Inc. Technical Report dapper-2010-1. 2010. Запись организации. Полный текст.

Роль: основа повсеместного инструментирования и вероятностного отбора. Проверены библиография и первичное описание. Отчёт не засчитывается в 19 рецензируемых публикаций.

21. Alves R. M., Nobre J. C., Wickboldt J. A. Dynamic Sampling for Telemetry in Microservices: A Reinforcement Learning and Entropy-Based Approach. 2026. arXiv:2609.31292v1. DOI: 10.48550/arXiv.2609.31292. Версия от 25.09.2026.

Роль: RADAR, современный кандидат адаптивного отбора на основе OpenTelemetry. По записи arXiv работа направлена в Journal of Network and Systems Management и находится на рецензировании. Проверена аннотация; заявления о ресурсах и редких шаблонах требуют проверки эксперимента и не используются как установленный результат.

8.3. Официальная документация и стандарт

Для [22–30] дата обращения: 29.10.2026. Документация описывает технологию, но не заменяет независимые экспериментальные доказательства.

22. OpenTelemetry. Components. Официальная документация. Страница.

Роль: разграничение API, SDK, инструментирования и Collector.

23. OpenTelemetry. Context Propagation. Официальная документация. Страница.

Роль: связь контекста между операциями и сервисами; основа согласованности сигналов.

24. OpenTelemetry. Sampling. Официальная документация. Страница.

Роль: определения раннего и позднего отбора и их сочетаний.

25. OpenTelemetry. Tracing SDK. Спецификация. Страница.

Роль: запись, отбор, обработчики спанов и экспорт; разделение расходов SDK.

26. OpenTelemetry. OTLP Specification. Спецификация протокола. Страница.

Роль: передача телеметрии и поведение экспорта, доставка и ошибки.

27. W3C. Trace Context. W3C Recommendation. 23 November 2021. Фиксированная редакция.

Роль: стандарт полей traceparent и tracestate для HTTP; не гарантирует наличия всех спанов.

28. OpenTelemetry. Scaling the Collector. Официальная документация. Страница.

Роль: масштабирование, насыщение и обработка с состоянием.

29. OpenTelemetry. Resiliency. Официальная документация. Страница.

Роль: очереди отправки, повторные попытки, постоянное хранение и условия потери данных.

30. OpenTelemetry Collector Contrib. Tail Sampling Processor. Документация компонента, ветка main. README.

Роль: политики, состояние, ожидание спанов и маршрутизация одной трассы в один экземпляр Collector. Ветка изменяется; для стенда требуется зафиксировать выпуск и конфигурацию.

9. Доступность и оставшиеся проверки

Для [1–4, 6, 8, 10] доступны полные первичные тексты; проверялись методы, постановки и относящиеся к выводам результаты. Это выборочное аналитическое чтение, а не постраничное изучение всех материалов. Числа из рисунков без подтверждения текстом не извлекались. Визуальная проверка деталей исследовательских графиков в эту версию не включена.

Для [5, 7] непосредственное открытие издательских страниц ограничено; использованы индексированные разделы первичного текста. Для [9] найдены индексированные разделы §5.1–5.3 PDF проекта, но непосредственно файл получить не удалось. Nezha включена с ограниченной аннотацией; полного чтения и цитирования её численных результатов нет.

Для [8] опубликованная библиография проверена по университетской записи: страницы 147–161, фамилия Hanson. Доступный конференционный текст имеет другую пагинацию и написание Hansson. Библиография следует опубликованной записи, ссылки на разделы относятся к прочитанному тексту. Для воспроизведения нужно сверить окончательную издательскую версию.

Для [11–21] глубина ознакомления указана в записи. ASTracer и RADAR включены для последующего чтения без неподтверждённых численных сравнений. Для [22–30] проверены официальные страницы; дата обращения не фиксирует версии SDK и Collector. Для W3C дана постоянная редакция.

Остаются пробелы: диагностика при потерях связанных сигналов; адаптивность при смене нагрузки; устойчивость Collector; асинхронные границы и поздние спаны. Нет оснований утверждать, что один метод решает все эти проблемы. Следующий шаг: получить полные тексты [5, 7, 9, 15], проверить эксперимент [21], затем выбрать общую диагностическую процедуру и сопоставимые условия измерения.

10. Термины и сокращения

- Наблюдаемость: возможность судить о внутреннем состоянии системы по доступным данным о её работе.
- Телеметрия: собираемые сведения о работе приложения и инфраструктуры.
- Метрика: числовое измерение; также временной ряд измерений.
- Лог: запись о событии с сообщением и сопутствующими полями.
- Распределённая трасса: связанные записи об исполнении запроса в нескольких компонентах.
- Спан (span): запись об операции с идентификаторами, временем и атрибутами.
- Инструментирование: подключение или добавление средств формирования телеметрии.
- API: программный интерфейс; SDK: библиотеки сбора, обработки и экспорта через этот интерфейс.
- OpenTelemetry: проект спецификаций и средств формирования, сбора и передачи телеметрии.

• Collector: компонент приёма, обработки и экспорта; OTLP: протокол передачи телеметрии OpenTelemetry.

• Контекст трассировки: сведения для связи операции с трассой и другими операциями; передача контекста: перенос этих сведений через границы выполнения и сервисов.

• Baggage: дополнительные пары ключ–значение, передаваемые с контекстом.

• Sampling: отбор части трасс; head sampling: ранний отбор; tail sampling: отбор по накопленным данным трассы.

• Адаптивный отбор: изменение правила или доли отбора по наблюдаемым условиям.

• Ретроспективный сбор: извлечение записанных данных после обнаружения симптома.

• Сериализация: преобразование данных в формат передачи или хранения.

• CPU: процессорные ресурсы; RPC: удалённый вызов процедуры.

• Пропускная способность: число обработанных запросов за единицу времени; успешные запросы учитываются отдельно.

• p95/p99: задержки, не превышенные соответственно в 95% и 99% измерений.

• Первопричина: исходный фактор нарушения; RCA: поиск первопричины, root cause analysis.

• SLA: соглашение об уровне обслуживания; SLO: целевое значение показателя; QoS: качество обслуживания.

• Причинность по Грейнджеру: предсказательная зависимость рядов; не достаточное доказательство физической причины.

• Контрфактическая оценка: оценка поведения системы при отсутствии предполагаемого фактора.

• Критический путь: цепочка зависимых операций, определяющая длительность исполнения.